



University of Computer Studies, Yangon (UCSY)

Introduction to Deep Learning

IS-212
(Data and Knowledge Mining)

Dr. Hsu Myat Mo
Faculty of Information Science, UCSY



Basic Concept

- Deep learning is based on artificial neural networks (ANNs) (or neural networks for short).
- A neural network is a set of connected input-output units in which each connection has a weight associated with it.
- During the learning phase, the network learns by adjusting the weights in order to predict the correct target values (e.g., class labels) of the input tuples.
- A neural network is made up of interconnected units.



Classification by Backpropagation

- A neural network learning algorithm
- A neural network: a set of connected input/output units in which each connection has a weight associated with it
- During the learning phase, the network learns by adjusting the weights so as to be able to predict the correct class label of the input tuples.
- Neural network learning is also referred to as connectionist learning due to the connections between units.



Neural Network as a Classifier

➤ Weakness

- long training times
- require a number of parameters that are typically best determined empirically such as the network topology or “structure.”
- poor interpretability



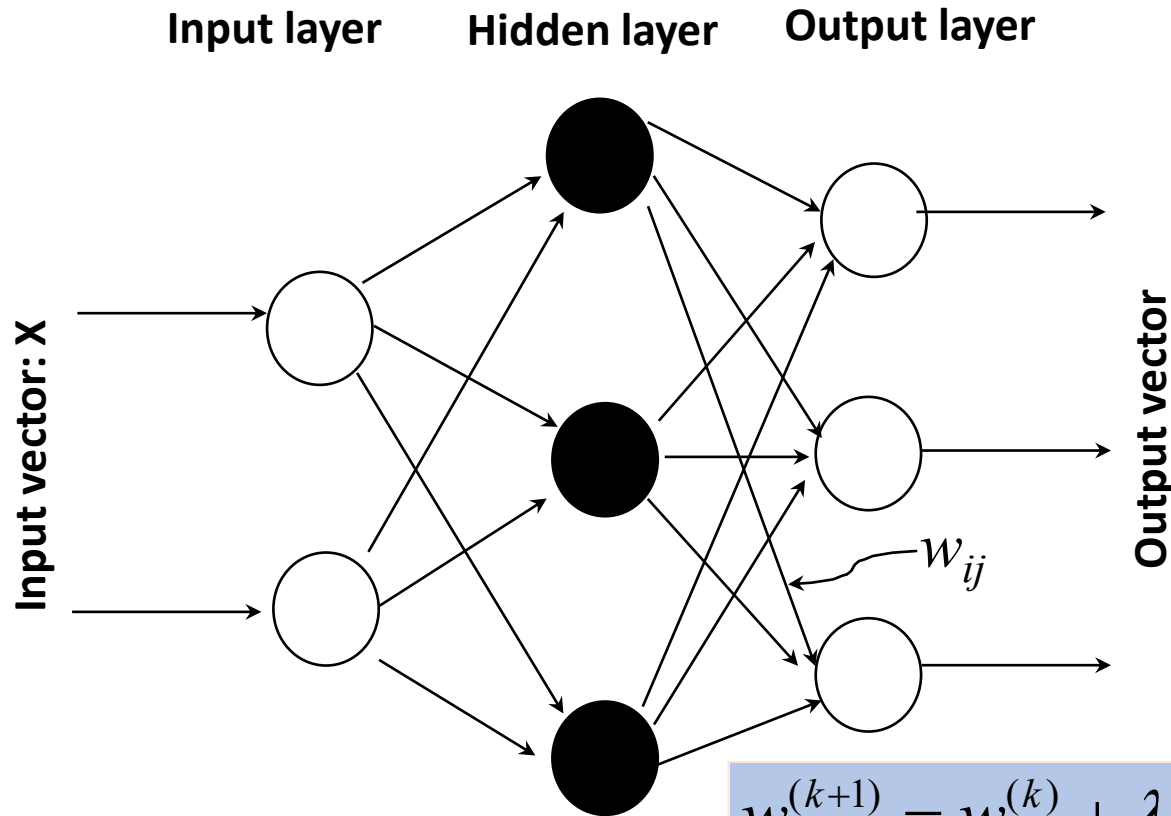
Neural Network as a Classifier

➤ Strength

- high tolerance of noisy data
- ability to classify patterns on which unknown data
- can be used when there may have little knowledge of the relationships between attributes and classes
- well suited for continuous-valued inputs and outputs
- successful on a wide array of real-world data
- Neural network algorithms are inherently parallel
- several techniques have been recently developed for rule extraction from trained neural networks



A Multilayer Feed-Forward Neural Network



The network is feed-forward: None of the weights cycles back to an input unit or to an output unit of a previous layer.

From a statistical point of view, networks perform *nonlinear regression*

Multilayer feed-forward networks, given enough hidden units and enough training samples, can closely approximate any function.

$$w_j^{(k+1)} = w_j^{(k)} + \lambda(y_i - \hat{y}_i^{(k)})x_{ij}$$



Defining a Network Topology

- Decide on the network topology: the number of units in the input layer, the number of hidden layers (if more than one), the number of units in each hidden layer, and the number of units in the output layer
- Normalize the input values for each attribute measured in the training tuples $[0.0, 1.0]$
- Once a network has been trained and its accuracy is not considered acceptable, it is common to repeat the training process with a different network topology or a different set of initial weights.
- Cross-validation techniques for accuracy estimation can be used to help decide when an acceptable network has been found.

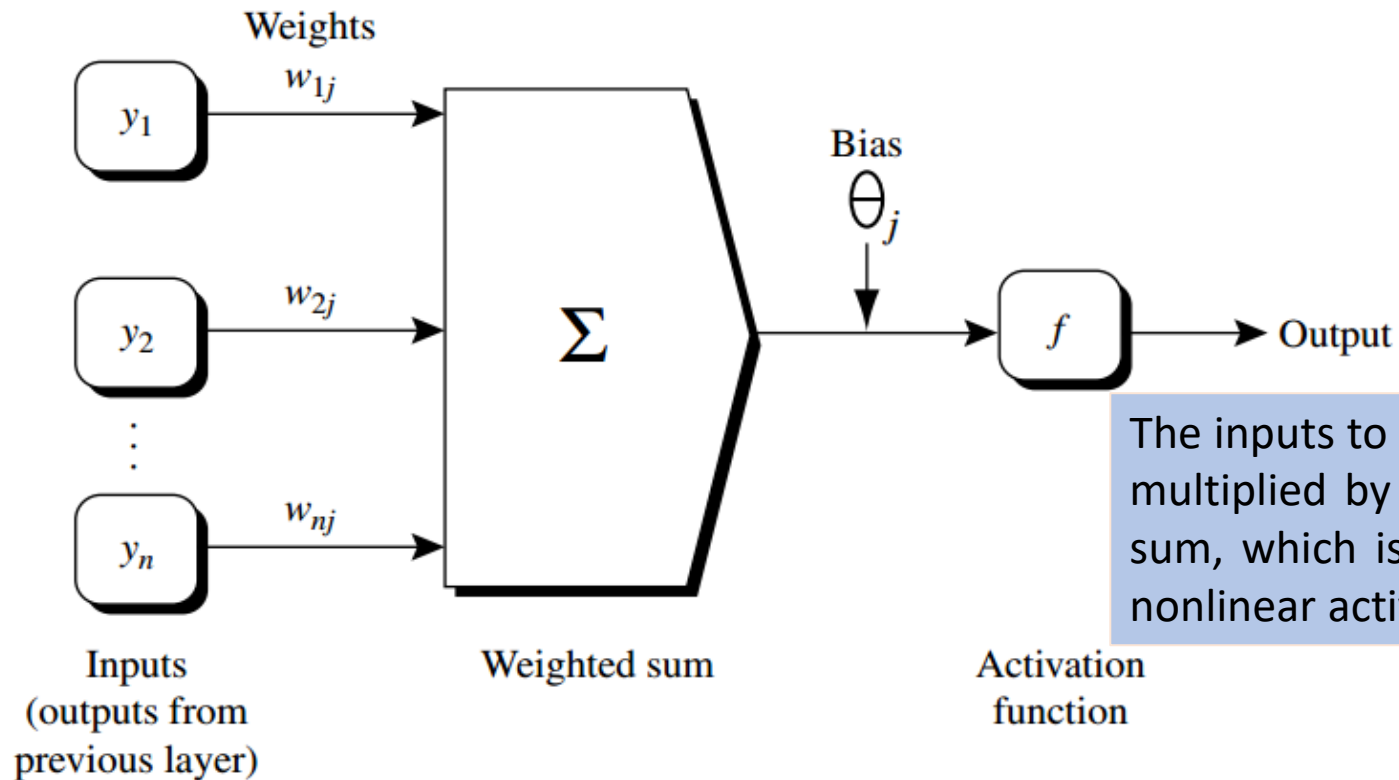


Backpropagation

- Iteratively process a set of training tuples & compare the network's prediction with the actual known target value
- For each training tuple, the weights are modified to minimize the mean squared error between the network's prediction and the actual target value
- Modifications are made in the “backwards” direction: from the output layer, through each hidden layer down to the first hidden layer, hence “backpropagation”
- Steps
 - Initialize the weights to small random numbers, associated with biases
 - Propagate the inputs forward (by applying activation function)
 - Backpropagate the error (by updating weights and biases)
 - Terminating condition (when error is very small, the percentage of tuples misclassified is very small, or reach the prespecified no. of epoch, etc.)



A Hidden/Output Layer Unit



The inputs to unit are outputs from the previous layer. They are multiplied by their corresponding weights to form a weighted sum, which is added to the bias associated with unit. Then a nonlinear activation function is applied to it.

Figure: A hidden/ output layer unit



Algorithm: Backpropagation. Neural network learning for classification or numeric prediction, using the backpropagation algorithm.

Input:

- D , a data set consisting of the training tuples and their associated target values;
- l , the learning rate;
- $network$, a multilayer feed-forward network.

Output: A trained neural network.

Method:

- (1) Initialize all weights and biases in $network$;
- (2) **while** terminating condition is not satisfied {
- (3) **for** each training tuple X in D {
- (4) // Propagate the inputs forward:
- (5) **for** each input layer unit j {
- (6) $O_j = I_j$; // output of an input unit is its actual input value
- (7) **for** each hidden or output layer unit j {
- (8) $I_j = \sum_i w_{ij} O_i + \theta_j$; // compute the net input of unit j with respect to the previous layer, i
- (9) $O_j = \frac{1}{1+e^{-I_j}}$; } // compute the output of each unit j
- (10) // Backpropagate the errors:
- (11) **for** each unit j in the output layer
- (12) $Err_j = O_j(1 - O_j)(T_j - O_j)$; // compute the error
- (13) **for** each unit j in the hidden layers, from the last to the first hidden layer
- (14) $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$; // compute the error with respect to the next higher layer, k
- (15) **for** each weight w_{ij} in $network$ {
- (16) $\Delta w_{ij} = (l) Err_j O_i$; // weight increment
- (17) $w_{ij} = w_{ij} + \Delta w_{ij}$; } // weight update
- (18) **for** each bias θ_j in $network$ {
- (19) $\Delta \theta_j = (l) Err_j$; // bias increment
- (20) $\theta_j = \theta_j + \Delta \theta_j$; } // bias update
- (21) } }



Cont'd

- Given a unit, j in a hidden or output layer, the net input, I_j , to unit j is $I_j = \sum_i w_{ij} O_i + \theta_j$,
- Each unit in the hidden and output layers takes its net input and then the function symbolizes the activation of the neuron represented by the unit.
- The **logistic**, or **sigmoid**, function is used.
- Given the net input I_j to unit j , then O_j , the output of unit j , is computed as applies an **activation** function to it, $O_j = \frac{1}{1 + e^{-I_j}}$.
- **Backpropagate the error:** The error is propagated backward by updating the weights and biases to reflect the error of the network's prediction. For a unit j in the output layer, the error Err_j is computed by

$$Err_j = O_j(1 - O_j)(T_j - O_j),$$

- The error of a hidden layer unit j is

$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk},$$



Cont'd

- The weights and biases are updated to reflect the propagated errors. Weights are updated by the following equations, where Δw_{ij} is the change in weight w_{ij} :

$$\Delta w_{ij} = (l) Err_j O_i.$$

$$w_{ij} = w_{ij} + \Delta w_{ij}.$$

- The variable l is the **learning rate**, a constant typically having a value between 0.0 and 1.0.
- Biases are updated by the following equations, where $\Delta \theta_j$ is the change in bias θ_j :

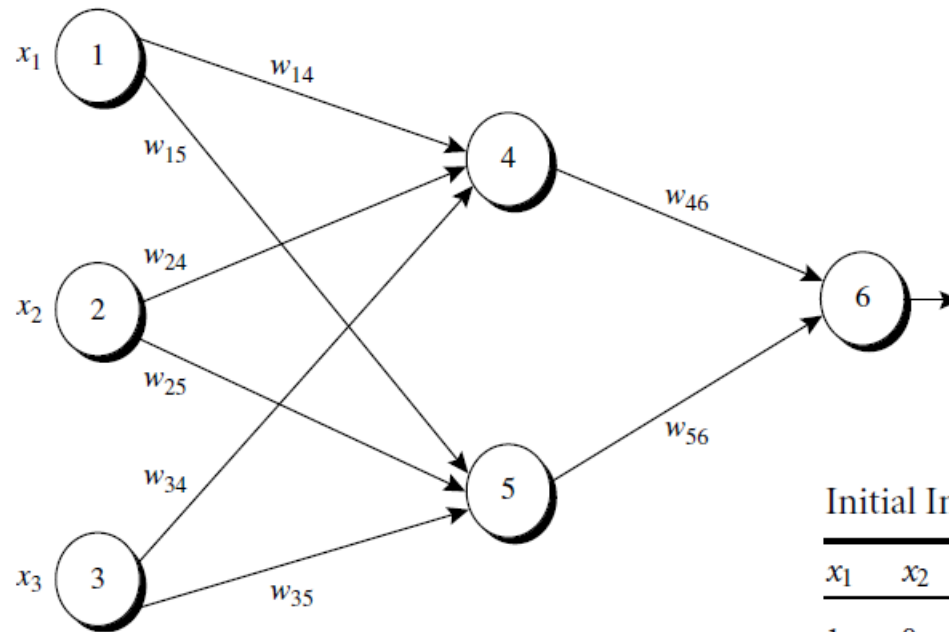
$$\Delta \theta_j = (l) Err_j.$$

$$\theta_j = \theta_j + \Delta \theta_j.$$



Sample calculations for learning by the backpropagation algorithm

- Let the **learning rate be 0.9**. The initial weight and bias values of the network are given in Table, along with the first training tuple, $X=(1, 0, 1)$, with a class label of **1**.



Initial Input, Weight, and Bias Values

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1



Cont'd

Initial Input, Weight, and Bias Values

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

Net Input and Output Calculations

Unit, j	Net Input, I_j	Output, O_j
4	$0.2 + 0 - 0.5 - 0.4 = -0.7$	$1/(1 + e^{0.7}) = 0.332$
5	$-0.3 + 0 + 0.2 + 0.2 = 0.1$	$1/(1 + e^{-0.1}) = 0.525$
6	$(-0.3)(0.332) - (0.2)(0.525) + 0.1 = -0.105$	$1/(1 + e^{0.105}) = 0.474$

Calculation of the Error at Each Node

Unit, j	Err $_j$
6	$(0.474)(1 - 0.474)(1 - 0.474) = 0.1311$
5	$(0.525)(1 - 0.525)(0.1311)(-0.2) = -0.0065$
4	$(0.332)(1 - 0.332)(0.1311)(-0.3) = -0.0087$

Calculations for Weight and Bias Updating

Weight or Bias	New Value
w_{46}	$-0.3 + (0.9)(0.1311)(0.332) = -0.261$
w_{56}	$-0.2 + (0.9)(0.1311)(0.525) = -0.138$
w_{14}	$0.2 + (0.9)(-0.0087)(1) = 0.192$
w_{15}	$-0.3 + (0.9)(-0.0065)(1) = -0.306$
w_{24}	$0.4 + (0.9)(-0.0087)(0) = 0.4$
w_{25}	$0.1 + (0.9)(-0.0065)(0) = 0.1$
w_{34}	$-0.5 + (0.9)(-0.0087)(1) = -0.508$
w_{35}	$0.2 + (0.9)(-0.0065)(1) = 0.194$
θ_6	$0.1 + (0.9)(0.1311) = 0.218$
θ_5	$0.2 + (0.9)(-0.0065) = 0.194$
θ_4	$-0.4 + (0.9)(-0.0087) = -0.408$

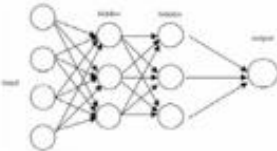
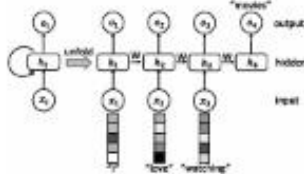
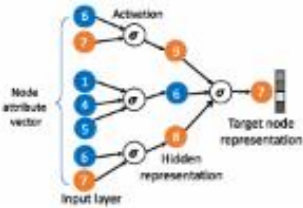


Terminating condition:

- All $1w_{ij}$ in the previous epoch are so small as to be below some specified threshold, or
- The percentage of tuples misclassified in the previous epoch is below some threshold, or
- A prespecified number of epochs has expired.



Overview of deep learning architecture

Data Type	Multidimensional Features: credit rating, account balance $x = (4.5, 500, 3, 5)$ #deposits, #withdraws	Grid 	Sequence $x = \text{"I love watching movies."}$	Graph 
DL Architecture	Feed-forward Network 	CNN 	RNN 	GNN 



Improve training of deep learning models

- Responsive activation functions
- Adaptive learning rate
- Dropout
- Pretraining- refers to the process of initializing the model parameters (the weights and bias values of a feed-forward neural network) in a suitable region.



End of Lecture!

hsumyatmo@ucsy.edu.mm